



UNIVERSIDAD
NACIONAL
DE COLOMBIA

Universidad Nacional de Colombia - sede Bogotá
Facultad de Ingeniería
Departamento de Ingeniería de Sistemas e Industrial
Curso: Arquitectura de Software

Estudiantes:

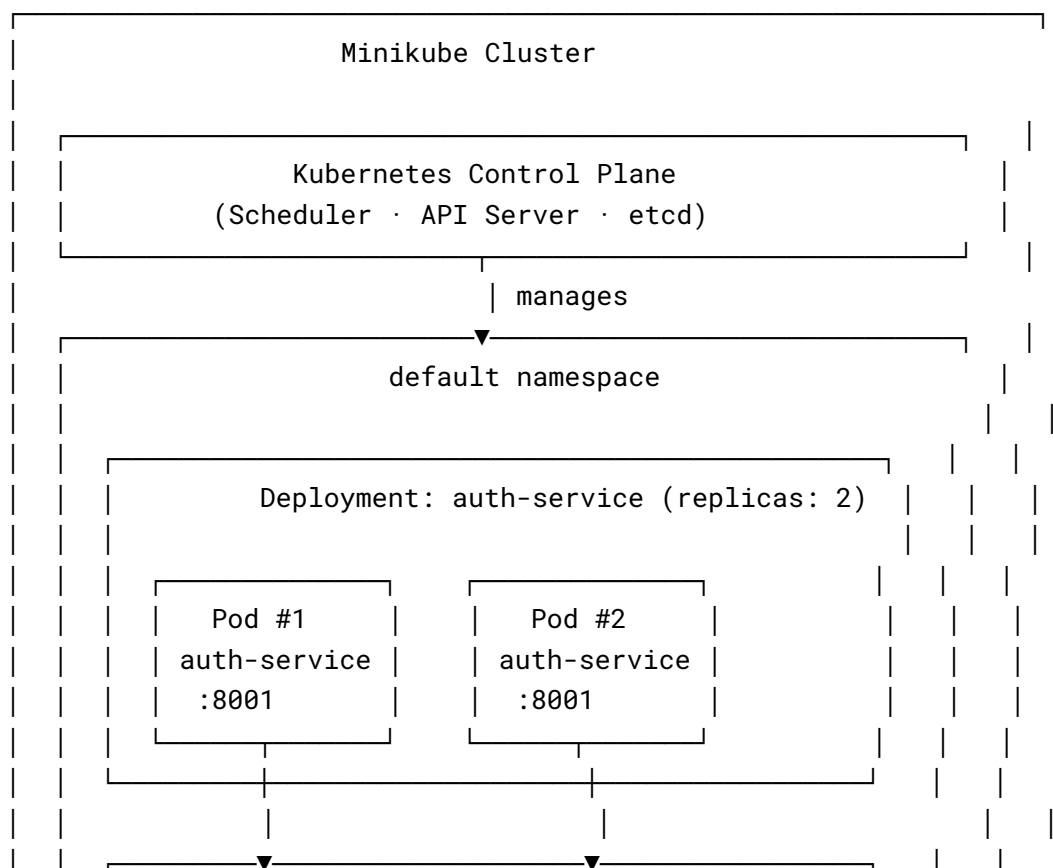
- John Alejandro Pastor Sandoval
- Andres Felipe Perdomo Uruburu
- Juan David Castañeda Cardenas
- Sara Isabel Ospina Valderrama

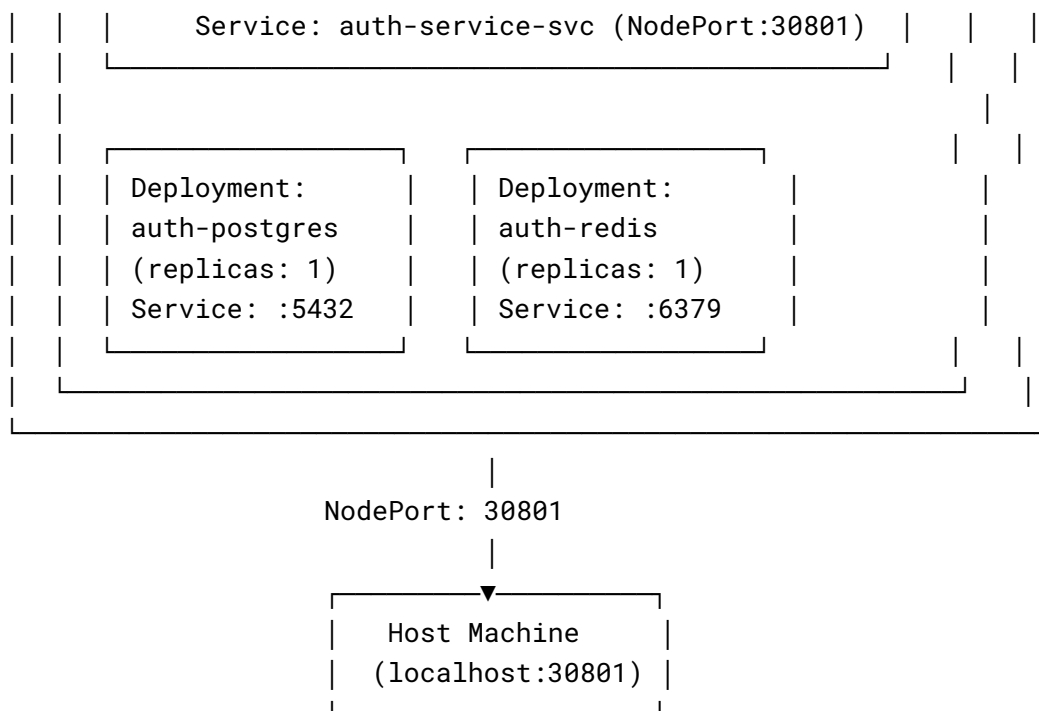
Parte A

Vista Arquitectónica

Cluster Pattern – Vista de Despliegue Kubernetes

El siguiente diagrama muestra cómo el `auth-service` fue desplegado como un Kubernetes Deployment con 2 réplicas dentro de un clúster local (Minikube), junto con sus dependencias internas (PostgreSQL y Redis).





Guía Técnica – Parte A: Cluster Pattern

Descripción del Patrón

El **Cluster Pattern** agrupa múltiples nodos para que actúen como un único sistema lógico. Kubernetes implementa este patrón organizando contenedores en **Pods**, agrupando Pods en **Deployments**, y exponiéndolos a través de **Services**. Esto proporciona:

- **Self-healing:** Kubernetes detecta pods caídos y los recrea automáticamente.
- **Escalado:** El número de réplicas se puede ajustar en tiempo real.
- **Load balancing:** El tráfico se distribuye entre todas las réplicas disponibles.

Tácticas de confiabilidad que soporta:

- Fault Detection (liveness y readiness probes)
- Redundant Spare (múltiples réplicas activas)
- Load Balancing (distribución automática de tráfico)

Tipo de Clúster Implementado

Se implementó un clúster **Active/Active**, donde todos los nodos (réplicas) procesan tráfico simultáneamente. Esto se eligió porque el **auth-service** es un servicio **stateless** – cada réplica puede atender cualquier petición sin depender del estado de las demás, ya que la sesión se almacena en Redis.

Componente Desplegado

auth-service – Servicio de autenticación (FastAPI/Python 3.12)

- Puerto interno: 8001
- Réplicas mínimas: 2
- Dependencias: PostgreSQL (auth_db), Redis (sesiones JWT)

Se eligió este componente porque es **stateless**, lo que facilita la replicación horizontal sin riesgo de inconsistencias de estado entre pods.

Pasos de Implementación

Paso 1 – Iniciar Minikube

```
bash
minikube start --driver=docker
```

Paso 2 – Apuntar Docker al daemon de Minikube y construir la imagen

```
bash
eval $(minikube docker-env)
docker build -t ecommerce-project-auth-service:latest \
  -f backend/auth_service/Dockerfile ./backend
```

⚠ **imagePullPolicy: Never** en el Deployment asegura que Kubernetes use la imagen local construida dentro de Minikube.

Paso 3 – Crear los manifiestos Kubernetes

Los manifiestos están en la carpeta **k8s/**:

```
k8s/
├── auth-deployment.yaml  # Deployment del auth-service (2 réplicas)
├── auth-service.yaml     # Service NodePort expuesto en :30801
└── auth-postgres.yaml   # PostgreSQL + Redis internos del clúster
```

Paso 4 – Aplicar los manifiestos

```
bash
kubectl apply -f k8s/auth-postgres.yaml
kubectl apply -f k8s/auth-deployment.yaml
kubectl apply -f k8s/auth-service.yaml
```

Paso 5 – Verificar el despliegue

```
bash
kubectl get pods
kubectl get svc
```

Salida esperada:

NAME	READY	STATUS	RESTARTS	AGE
auth-postgres-8d5db755f-9x6gp	1/1	Running	0	5m
auth-redis-89d444447-gk2c5	1/1	Running	0	5m
auth-service-577859fc9c-h5j55	1/1	Running	0	5m
auth-service-577859fc9c-pskc7	1/1	Running	0	5m

Snippets de Configuración

k8s/auth-deployment.yaml

yaml

apiVersion: apps/v1

kind: Deployment

metadata:

name: auth-service

labels:

app: auth-service

spec:

replicas: 2

selector:

matchLabels:

app: auth-service

template:

metadata:

labels:

app: auth-service

spec:

containers:

- name: auth-service

image: ecommerce-project-auth-service:latest

imagePullPolicy: Never

ports:

- containerPort: 8001

env:

- name: DATABASE_URL

value:

"postgresql+asyncpg://auth_user:auth_password@auth-postgres:5432/auth_db"

- name: REDIS_URL

value: "redis://auth-redis:6379/0"

- name: JWT_SECRET

value: "replace_with_a_secure_secret_min_32_chars!!"

- name: RABBITMQ_HOST

value: "localhost"

livenessProbe:

httpGet:

```
    path: /health
    port: 8001
    initialDelaySeconds: 20
    periodSeconds: 20
  readinessProbe:
    httpGet:
      path: /health
      port: 8001
    initialDelaySeconds: 15
    periodSeconds: 10
```

k8s/auth-service.yaml

```
yaml
apiVersion: v1
kind: Service
metadata:
  name: auth-service-svc
spec:
  type: NodePort
  selector:
    app: auth-service
  ports:
    - port: 80
      targetPort: 8001
      nodePort: 30801
```

Adaptaciones específicas del proyecto:

- `imagePullPolicy`: `Never` para usar imagen local de Minikube.
- Variables de entorno ajustadas al naming del `auth-service` (`DATABASE_URL`, `REDIS_URL`, `JWT_SECRET`).
- PostgreSQL y Redis desplegados dentro del clúster para evitar dependencias externas.
- Liveness y Readiness probes apuntando al endpoint `/health` del `auth-service`.

Evidencia de Self-Healing

Se eliminó manualmente un pod y Kubernetes lo recreó automáticamente para mantener las 2 réplicas:

```
bash
# Eliminar un pod manualmente
kubectl delete pod auth-service-577859fc9c-9csm8

# Observar recreación automática
kubectl get pods --watch
```

Salida observada:

NAME	READY	STATUS	RESTARTS	AGE
auth-service-577859fc9c-9csm8	0/1	Terminating	4	3m42s
auth-service-577859fc9c-h5j55	1/1	Running	0	3m42s
auth-service-577859fc9c-pskc7	0/1	Running	0	2s
← nuevo pod				
auth-service-577859fc9c-pskc7	1/1	Running	0	26s
← listo				

Kubernetes detectó que el número de réplicas cayó a 1 y creó un nuevo pod (**pskc7**) automáticamente en segundos.

Evidencia de Escalado

```
bash
# Escalar a 4 réplicas
kubectl scale deployment auth-service --replicas=4

# Verificar
kubectl get pods
```

Salida observada:

NAME	READY	STATUS	RESTARTS
auth-postgres-8d5db755f-9x6gp	1/1	Running	0
5m12s			
auth-redis-89d444447-gk2c5	1/1	Running	0
5m12s			
auth-service-577859fc9c-7nfvh	0/1	ContainerCreating	0
0s ← nuevo			
auth-service-577859fc9c-h5j55	1/1	Running	0
3m19s			
auth-service-577859fc9c-ps5l4	0/1	ContainerCreating	0
0s ← nuevo			
auth-service-577859fc9c-pskc7	1/1	Running	0
92s			

El clúster pasó de 2 a 4 réplicas sin interrumpir el servicio.

```

astarteaz@ISA:/mnt/c/Users/ASUS/Documents/GitHub/ecommerce-project$ kubectl
get podsds
NAME                                READY   STATUS    RESTARTS   AGE
auth-postgres-8d5db755f-9x6gp      1/1     Running   0           2m45s
auth-redis-89d444447-gk2c5         1/1     Running   0           2m45s
auth-service-577859fc9c-9csm8      1/1     Running   3 (89s ago) 2m45s
auth-service-577859fc9c-h5j55      1/1     Running   0           52s
astarteaz@ISA:/mnt/c/Users/ASUS/Documents/GitHub/ecommerce-project$ kubectl
delete pod auth-service-577859fc9c-9csm8
kubectl get pods --watch
pod "auth-service-577859fc9c-9csm8" deleted from default namespace
NAME                                READY   STATUS    RESTARTS   AGE
auth-postgres-8d5db755f-9x6gp      1/1     Running   0           3m42s
auth-redis-89d444447-gk2c5         1/1     Running   0           3m42s
auth-service-577859fc9c-h5j55      1/1     Running   0           109s
auth-service-577859fc9c-pskc7      0/1     Running   0           2s
auth-service-577859fc9c-pskc7      1/1     Running   0           26s

```

```

astarteaz@ISA:/mnt/c/Users/ASUS/Documents/GitHub/ecommerce-project$ kubectl
scale deployment auth-service --replicas=4=4
kubectl get pods
deployment.apps/auth-service scaled
NAME                                READY   STATUS             RESTARTS   AGE
auth-postgres-8d5db755f-9x6gp      1/1     Running             0           5m12s
auth-redis-89d444447-gk2c5         1/1     Running             0           5m12s
auth-service-577859fc9c-7nfvh      0/1     ContainerCreating   0           0s
auth-service-577859fc9c-h5j55      1/1     Running             0           3m19s
auth-service-577859fc9c-ps5l4      0/1     ContainerCreating   0           0s
auth-service-577859fc9c-pskc7      1/1     Running             0           92s
astarteaz@ISA:/mnt/c/Users/ASUS/Documents/GitHub/ecommerce-project$ kubectl
scale deployment auth-service --replicas=2
deployment.apps/auth-service scaled
astarteaz@ISA:/mnt/c/Users/ASUS/Documents/GitHub/ecommerce-project$ |

```

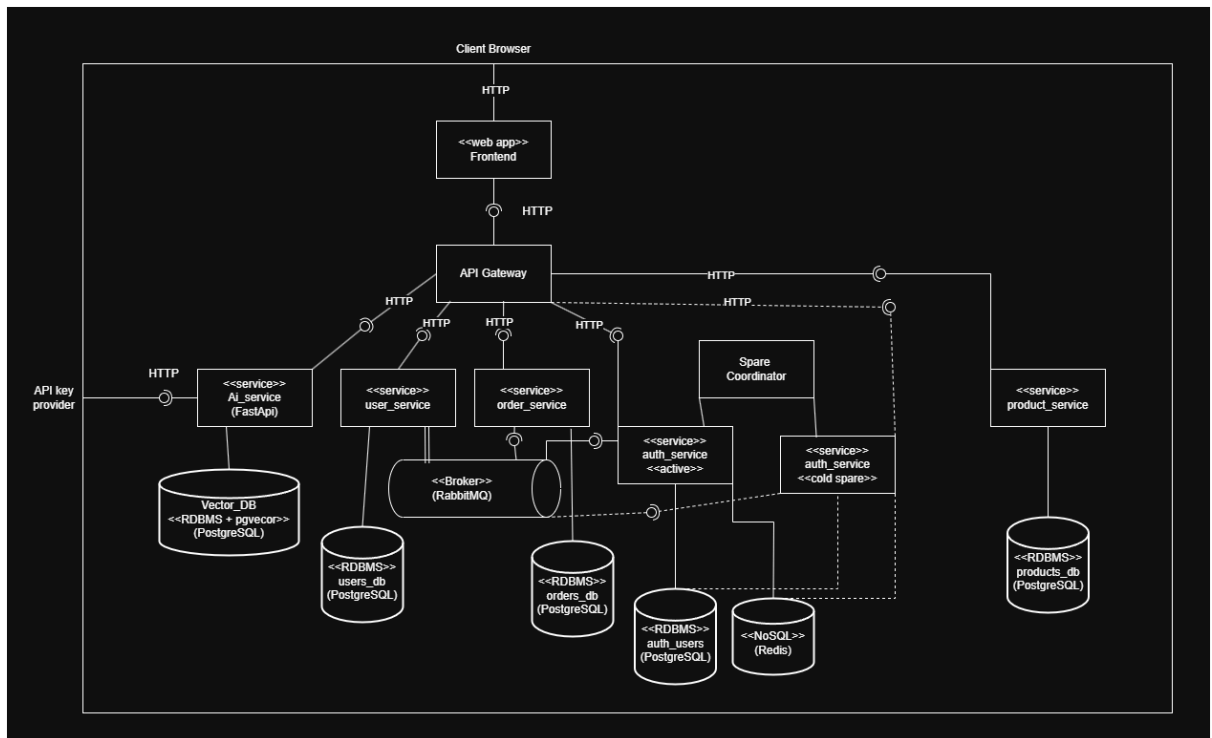
Parte B

Descripción del Escenario:

Source	Múltiples usuarios de la plataforma (carga masiva estimada en 5,000 usuarios concurrentes)
Stimulus	Las peticiones concurrentes de registro e inicio de sesión saturan el servicio de autenticación principal (auth_service), provocando una caída total del proceso (crash) y la indisponibilidad del servicio.
Artifact	El servicio de autenticación
Environm ent	Operación pico, donde hay una gran cantidad de usuarios haciendo uso del sistema (fin de semana por la tarde)

Response	El sistema detecta la caída del auth_service en un tiempo menor a 20 segundos, activa la cold spare del servicio en menos de un minuto
Response measure	Los tiempo de reacción a la hora de identificar el fallo debe ser de segundos y menor al límite acordado, el tiempo que tarda en activar la copia se debe de medir en segundos y no debe de superar el minuto (60 segundos)

Diagrama de componentes y conectores:



Enlace de la imagen: [CyCLab6.drawio.png](#)

Guia Tecnica:

Pattern description

El patrón utilizado fue **Cold Redundancy**, consiste en tener una copia inactiva de uno de los servicios (auth_service) y en el momento que la instancia principal de este servicio falle, esto es detectado por el **Spare Coordinator**, quien activa la **cold spare** del servicio para su funcionamiento.

En nuestro caso, el auth_service no guarda ningún tipo de información, todo lo guarda en bases de datos, el servicio no guarda ningún tipo de estado, por esta razón es que no se implementó un sistema de "checkpoints".

Implementation steps

Para su implementación se realizó la siguiente secuencia de pasos:

- Modificar el Docker-compose, asignado un rol de "active" a contenedor del auth_service.
- Crear el contenedor **auth-service-cold**, este contenedor es casi idéntico al auth_service, con la diferencia que tiene un rol de "spare" y funciona con un puerto distinto.
- Crear un coordinador que hace health checks al servicio de autenticación usando comandos del docker.
- Implementar el Contenedor del coordinador.
- Modificar el endpoint de Health del auth_service.
- Agregar un endpoint al auth_service para poder manejar los roles planteados en los contenedores.

Configuration or code snippets

A continuación están el código de los contenedores agregados y modificados, en ellos se representa un resumen de las acciones hechas:

None

```
auth-service:
  build:
    context: ./backend
    dockerfile: auth_service/Dockerfile
  container_name: ecommerce_auth_service
  restart: unless-stopped
  depends_on:
    auth-postgres:
      condition: service_healthy
    redis:
      condition: service_healthy
  environment:
    DATABASE_URL: ${AUTH_DATABASE_URL}
    APP_NAME: ${AUTH_APP_NAME:-Auth Service Active}
    ROLE: active
    REDIS_URL: ${REDIS_URL:-redis://redis:6379/0}
  command: uvicorn auth_service.main:app --host 0.0.0.0 --port 8001

auth-service-cold:
  build:
    context: ./backend
    dockerfile: auth_service/Dockerfile
  container_name: ecommerce_auth_service_cold
  restart: unless-stopped
  depends_on:
    auth-postgres:
      condition: service_healthy
    redis:
```

```

        condition: service_healthy
environment:
  DATABASE_URL: ${AUTH_DATABASE_URL}
  APP_NAME: ${AUTH_APP_NAME:-Auth Service Cold Backup}
  ROLE: spare
  REDIS_URL: ${REDIS_URL:-redis://redis:6379/0}
command: uvicorn auth_service.main:app --host 0.0.0.0 --port 8002

spare-coordinator:
  build:
    context: ./coordinator
    dockerfile: Dockerfile
  container_name: ecommerce_spare_coordinator
  restart: unless-stopped
  volumes:
    - /var/run/docker.sock:/var/run/docker.sock
  depends_on:
    auth-service:
      condition: service_started

```

Evidence of failover

Para este paso se desactivo el contenedor del auth_service para verificar como el coordinador lo maneja, al desactivar el servicio el coordinador nos muestra los siguientes mensajes dentro del log:

auth-service-cold	INFO: 172.20.0.14:38862 - "POST /activate HTTP/1.1" 200 OK
spare-coordinator	[COORDINATOR] Monitoring active service on port 8001... [COORDINATOR] Active service unreachable. [COORDINATOR] Active service unreachable. [COORDINATOR] Active service unreachable. [COORDINATOR] Active service down. Triggering failover... [COORDINATOR] Spare instance activated successfully.

Aquí se evidencia cómo cambia el rol del auth-service-cold y los mensajes del Coordinador, el coordinador trata de contactar con el servicio tres veces, si en las tres falla entonces activa la cold spare, en la captura se evidencia cada uno de los mensajes demostrando que trató de conectarse al servicio 3 veces, al no recibir respuesta se inicia el proceso para activar la copia de reserva.

Recommendations

- Identificar si el servicio guarda algún tipo de información o estado, en caso de guardarla es importante crear una serie de "checkpoints" y asegurar que la copia se active con el último checkpoint guardado.
- Si el servicio es propenso a fallar considerar implementar más copias. Si se ve necesario, es mejor implementar otro tipo de patrones (como Passive Redundancy o Active Redundancy).